

PriceTracker – Suivi de l'Inflation consommateur

Projet annuel – Mastère Intelligence Artificielle & Big Data

ESGI 5 IABD 2 – Année académique 2025-2026



Made with GAMMA

Vision : Mesurer l'Inflation perçu

PriceTracker est une plateforme crowdsourcée qui mesure l'inflation **réelle et vécue** par les consommateurs français. Son originalité repose sur la combinaison de deux sources de données :

Dataset Public Open Prices

Prix au niveau EAN, géolocalisés, datés, mis à jour quotidiennement (HuggingFace).

Tickets de Caisse Utilisateurs

Tickets uploadés par les utilisateurs, traités par un pipeline OCR/NLP.

Ces données sont croisées avec un **catalogue produit normalisé** (Open Food Facts) pour calculer des indices d'inflation personnalisés, régionaux et nationaux, comparés aux chiffres officiels de l'INSEE.

Personas Clés

Nous avons identifié deux personas principales pour PriceTracker, chacune avec des besoins spécifiques liés au suivi de l'inflation.



Mère de famille (35-45 ans)

Budget serré, achète chez Leclerc/Lidl. Cherche à réduire sa facture de 10-15% et reçoit des alertes sur les hausses de prix.

Marie, 38 ans, employée administrative avec 2 enfants, gagne environ 2 000€/mois. Elle photographie ses tickets pour suivre l'évolution de ses produits habituels (lait, pain, hygiène) et cherche des substituts moins chers ainsi que des alertes de hausse.



Jeune actif (25-35 ans)

Étudiant ou salarié. Souhaite un suivi hebdomadaire et trouver les prix les moins chers dans sa ville.

Flux Utilisateur Principal

Parcours d'un utilisateur, de l'upload d'un ticket de caisse à la réception d'alertes personnalisées et la consultation de l'observatoire public.

1

Upload

L'utilisateur photographie son ticket, qui est directement uploadé vers GCS et déclenche le Worker OCR.

2

Extraction & Résolution

Preprocessing image, inférence OCR, parsing structuré et résolution EAN via NLP matching.

3

Feedback Utilisateur

Résultat présenté à l'utilisateur pour feedback et correction, alimentant les alias produits.

4

Calcul périodique (2* hebdo ou Nightly)

Job BigQuery pour calculer les indices d'inflation (personnel, régional, national) et détecter les anomalies de prix.

5

Alerte

Détection des hausses sur les produits habituels et envoi de notifications push.

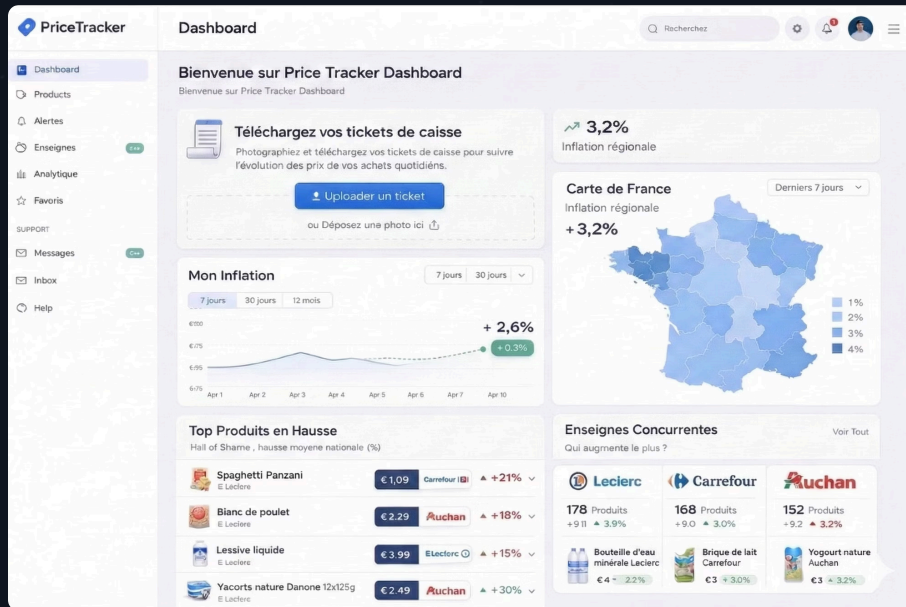
6

Observatoire

Suivi de la consommation personnelle et analyse de l'inflation en France.

Fonctionnalités Utilisateur & Composantes IA

WebApp (Next.js)



- Upload de tickets de caisse avec extraction automatique.
- Validation/correction des articles mal reconnus.
- Dashboard personnel : inflation, évolution dans le temps.
- Carte choroplèthe France : inflation par département.
- Classement enseignes, "Hall of Shame" des produits en hausse.
- Recommandations d'économies personnalisées.

Composantes IA & Data



Pipeline OCR - (Modèle OCR)

Traitement des photos de tickets (preprocessing, extraction texte via modèle custom, parsing structuré, score de confiance).



Résolution EAN ()

Mapper texte OCR bruité vers EAN canonique (NLP matching, similarité vectorielle, feedback loop utilisateur).



Recommandations Économies

Comparaison inter-enseignes, substituts par catégorie et sémantiques.



Détection Anomalies Prix

Filtrage des prix aberrants via Z-score SQL pour garantir la qualité des indices.

Calcul des Indices d'Inflation

Les indices sont calculés nuitamment dans BigQuery à partir des données Silver validées. Nous utilisons une formule d'indice de Laspeyres pondérée par la fréquence d'achat, standard et académiquement défendable.



Indice Personnel

Panier réel de l'utilisateur (produits achetés ≥ 2 fois sur 6 mois), variation N vs N-1.



Indice Régional

Agrégation par département.



Indice National

Agrégation France entière

Sources de Données

Open Prices (HuggingFace)

Datasets: openfoodfacts/open-prices

like 6

Following Open Food Facts 133

Dataset card

Split (1)

prices · 220k rows

Search this dataset

id	type	product_code	product_name	category_tag	labels_tags	origins_tags	price	price_is_discounted
int64	string	string	string	string	list	list	unknown	bool
42,629	PRODUCT	3566070283484	null	null	null	null	"1.250"	false
42,630	PRODUCT	3566070206469	null	null	null	null	"1.990"	false
39,637	PRODUCT	3123930711758	null	null	null	null	"3.600"	false
39,573	PRODUCT	3500390010247	null	null	null	null	"19.950"	false
33,784	PRODUCT	8033976484743	null	null	null	null	"2.990"	false
39,518	PRODUCT	3766020507916	null	null	null	null	"3.130"	false
39,625	PRODUCT	3220035229611	null	null	null	null	"2.770"	false
39,622	PRODUCT	3770014111777	null	null	null	null	"2.040"	false
39,626	PRODUCT	3250392335137	null	null	null	null	"2.070"	false
39,610	PRODUCT	3280220908073	null	null	null	null	"1.480"	false
39,633	PRODUCT	3250390523970	null	null	null	null	"1.090"	false
39,624	PRODUCT	3250392404499	null	null	null	null	"2.090"	false
39,627	PRODUCT	3250391980253	null	null	null	null	"1.790"	false

- Format : Parquet, mis à jour quotidiennement (~1 MB/jour incrémental).
- Contient des données PRODUCT (EAN) et CATEGORY (produits bruts/vrac).

Tickets de Caisse Utilisateurs

Carrefour market

Paris Lyon

Tel : 01 55 78 83 70

Ouvert du lundi au samedi

de 8h30 à 22h00

le dimanche de 9h00 à 19h30

TVA	DESCRIPTION	QTE x P.U.	MONTANT TTC
6	*1/2 EPINARDS HACHE		1,25€
6	*20CL CREM.FRAICH E		1,39€
6	*3X1/4 CHAMPIG. PIE		1,99€
6	*FLAN X 1		0,75€
6	*FLT 1KG PDT SIMPL		0,99€
2	*POELEE COURG BUTT		3,48€
6	*X4 0% VRAI BIO		1,38€
6	*X4 0% VRAI BIO		1,38€
8 ARTICLE(S) TOTAL A PAYER			12,61€
CB EMV SANS CONTACT EUR			12,61€
Taux	TVA.	Hors Taxe	
2> 5,50	0,18	3,30	
6> 5,50	0,47	8,66	
Totaux:	0,65	11,96	
8949	0613	032	005582 28/10/2024 17:39:32

- Photos uploadées via l'application web/mobile.
- Données extraites : enseigne, date, articles (texte brut → EAN), prix, quantités.
- Source primaire pour les contributions communautaires récentes.

Open Food Facts (OFF) API

openFOOD facts

Chercher un produit

Découvrir Contribuer Producteurs

INSTALLER L'APP

Produit

Vos critères Santé Environnement Signaler un problème Contribution

Prince Goût Chocolat – LU – 300 g

Cde-barre: 762220449283 (EAN / EAN-13)

Dénomination générique: BISCUITS FOURRÉS (55%) PARFUM CHOCOLAT

Quantité: 300 g

Emballage: Paquet, enEtu en carton, enFilm en plastique, enHôpe film-packet

Marques: LU

Catégories: Snacks, Petit-déjeuners, Snacks sucrés, Biscuits et gâteaux, Biscuits sucrés & biscuits apéritifs, Biscuits, Biscuits fourrés, Biscuits sandwich

Labels, certifications, récompenses: Labels de distributeurs, enCharte LU Harmony, Triman

Lien vers la page du produit sur le site officiel du fabricant: https://www.lu.fr/prince

Magasins: Carrefour Market, Magasins U, Auchan, Intermarché, Carrefour, Casino, Cora, Btl, carrefour.fr, Netto, Bannette, E.Leclerc

Pays de vente: Algérie, Belgique, France, Polynésie française, Allemagne, Guadeloupe, Hongrie, Luxembourg, Martinique, Maroc, Nouvelle-Calédonie, La Réunion, Espagne, Suisse, États-Unis

Correspondance avec vos préférences

Nutri-Score E

Aliments ultra-transformés

Green-Score D

- Enrichissement du catalogue à partir de l'EAN.
- Données récupérées : nom canonique, marque, catégorie normalisée, Nutriscore, Ecoscore.

Catalogue Produits – Brique Centrale

Le catalogue est le référentiel dont dépendent toutes les autres composantes. Sans lui, les EAN sont opaques et incomparables.

Structure Logique

`products` (ean, name_canonical, brand, category_1/2/3, unit_reference, nutriscore, last_synced_at)
`product_aliases` (raw_text, ean, confidence, validated_by_user)

Catalogage Produits Bruts

Les produits de type CATEGORY (bruts/vrac) sont comparables via `category_tag`, `labels_tags` et `origins_tags` sans passer par l'EAN.

Comparabilité & Substituts

Deux produits sont comparables s'ils partagent la même `category_l3` et la même unité de référence. La recherche de substituts sémantiques utilise des embeddings produits (pgvector).

INFRA



Cloud Run

Exécution serverless des workers conteneurisés (Docker) pour une scalabilité et une efficacité maximales.



Cloud Scheduler

Orchestration horaire précise de tous les processus, assurant une exécution fiable et ponctuelle de 03h00 à 07h00 UTC.



BigQuery

Sert de source unique de vérité, centralisant toutes les données enrichies dans ses tables Silver et Gold pour une analyse cohérente.



Cloud SQL

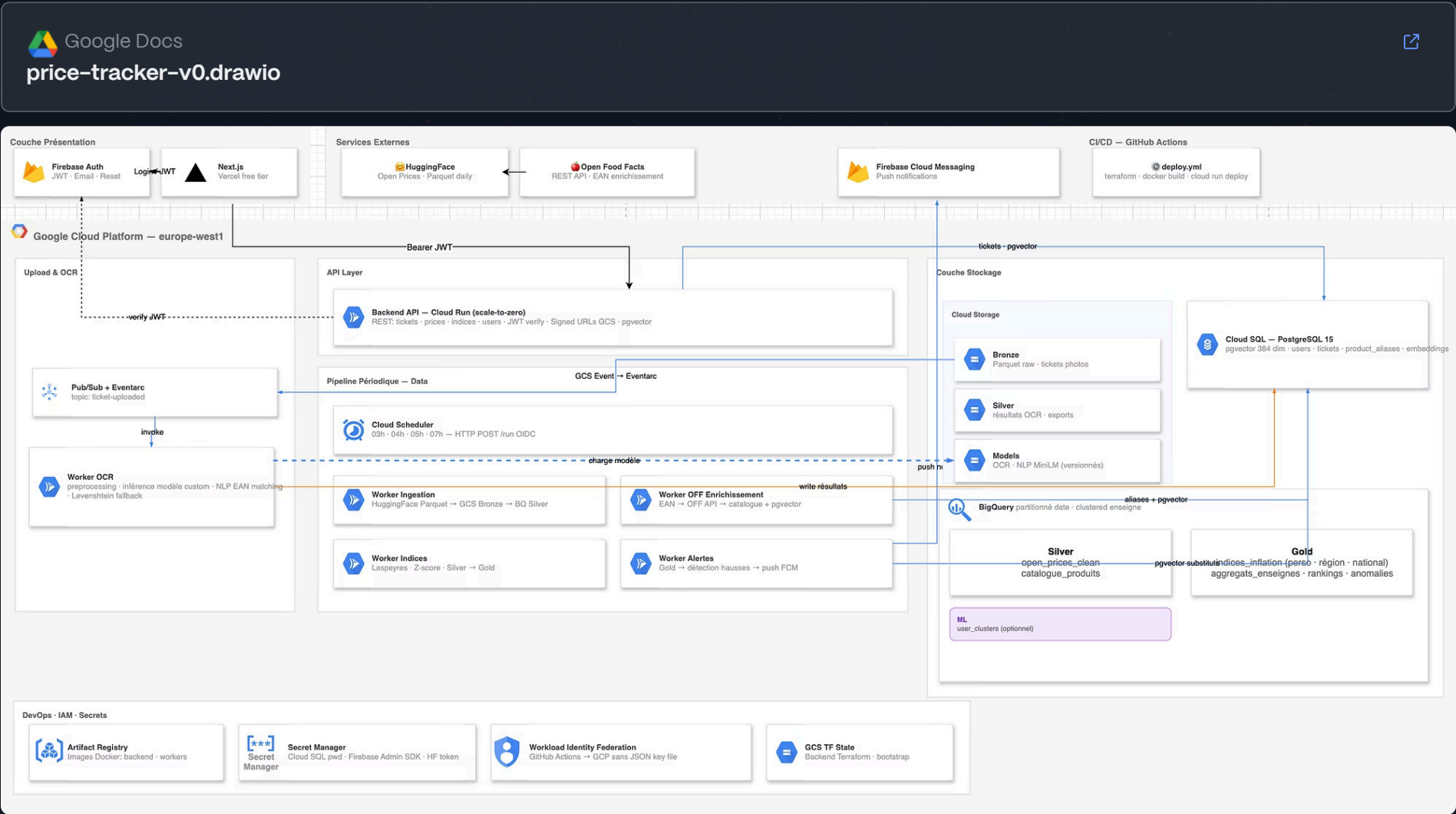
Pour les données transactionnels



Terraform

Gestion complète de l'infrastructure en tant que code (IaC) pour le projet, les datasets, les buckets pour reproductibilité et auditabilité.

SCHÉMA TECHNIQUE INFRA



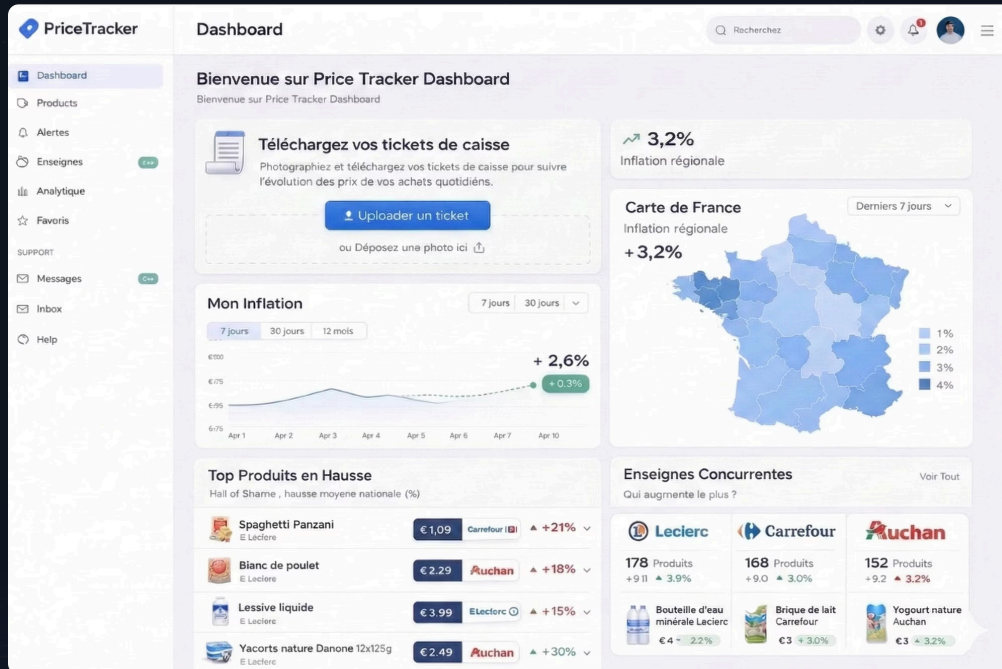
Architecture Technique – Synthèse

Infrastructure intégralement sur **Google Cloud Platform (GCP)**, provisionnée en **Terraform** pour la portabilité.

Service	Rôle
Cloud Run	Backend API (scale-to-zero), Workers data et OCR
Cloud SQL (PG 15)	Données opérationnelles : users, tickets, aliases, embeddings produits (pgvector)
BigQuery	Data warehouse : Open Prices (bronze→silver→gold), catalogue, indices, agrégats observatoire
Cloud Storage (GCS)	Images tickets, fichiers Parquet archivés, modèles ML
Pub/Sub + Eventarc	Déclenchement asynchrone du Worker OCR à l'upload
Firebase Auth	Authentication managée (JWT, email/password, reset)
Firebase Cloud Msg	Push notifications alertes
Artifact Registry	Images Docker des services Cloud Run
Secret Manager	Credentials (Cloud SQL, Firebase Admin SDK...)

Frontend & Orchestration Data

Frontend



- **Next.js** déployé
- Observatoire public et dashboard utilisateur.

Orchestration Data (Bronze → Silver → Gold)

L'orchestration est gérée par Cloud Scheduler, appelant des Cloud Run Services.

03h00 : Worker Ingestion

HuggingFace Parquet →
GCS → BQ Silver

05h00 : Worker Indices

Silver → calcul indices →
BQ Gold + INSEE

04h00 : Worker OFF

EAN → OFF API →
catalogue + embeddings

07h00 : Worker Alertes

Gold → détection hausses
→ push FCM

Stockage des Données

Les données sont réparties entre **Cloud SQL** pour les transactions opérationnelles, **BigQuery** pour l'analytique et l'historisation, et **Cloud Storage** pour les fichiers bruts.

Cloud SQL (PostgreSQL)

Table	Cas d'usage concret
users	Profil utilisateur, préférences, département. Complète Firebase Auth.
tickets	Métadonnées des tickets uploadés : statut, lien GCS, enseigne, date.
prix_extraits	Lignes articles brutes des tickets, avec prix, confiance OCR, et validation utilisateur.
product_aliases	Mapping "texte brut" → EAN canonique, enrichi par le feedback utilisateur.
products (pgvector)	Embeddings des fiches produit pour les recommandations sémantiques.
user_basket_history	Historique des EAN achetés régulièrement par l'utilisateur pour l'indice personnel.
notification_prefs	Seuils d'alerte, fréquence et enseignes favorites par utilisateur.

BigQuery

Plateforme d'entrepôt de données pour les flux Bronze, Silver et Gold.

Dataset Silver

Table	Contenu	Alimenté par
open_prices_clean	Entrées Open Prices filtrées France, dédupliquées et partitionnées par date.	Worker Ingestion (03h00)
catalogue_produits	Fiches EAN (nom, marque, catégorie L1/L2/L3, nutriscore, unité de référence).	Worker OFF (04h00)

Dataset Gold

Table	Contenu	Alimenté par
indices_inflation	Indices de Laspeyres (national, régional).	Worker Indices (05h00)
aggregats_enseignes	Prix moyen par produit × enseigne × semaine.	Worker Indices (05h00)
rankings_produits	Top hausses du mois, Hall of Shame.	Worker Indices (05h00)
anomalies_detected	Prix aberrants exclus des calculs d'indices.	Worker Indices (05h00)

Cloud Storage

GCS Bronze/

- `/open-prices/date=YYYY-MM-DD/snapshot.parquet` : Archives brutes des données HuggingFace.
- `/tickets/raw/user_id/uuid.jpg` : Images brutes des tickets uploadés par les utilisateurs.

GCS Silver

- `/tickets/processed/uuid_result.json`

Catégorisation et Embeddings : Le Processus



Détection EAN

Un nouveau code EAN est identifié dans la table `open_prices_clean` (BigQuery Silver).



Activation Worker OFF

Le processus est déclenché par le `Worker OFF` (chaque jour à 04h00).



Appel OFF API

L'API Open Food Facts est interrogée avec le nouvel EAN.



Récupération des Données

Les informations du produit (nom, marque, Nutriscore, PNNS) sont extraites.



Normalisation de l'Unité

L'unité de référence du produit (ex: 1L, 1kg) est normalisée.



Mise à Jour Catalogue BQ

Les données enrichies sont insérées ou mises à jour dans la table `catalogue_produits` (BigQuery Silver).



Génération d'Embeddings

Un embedding vectoriel est créé à partir des attributs du produit (nom, marque, catégorie).



Stockage Embeddings SQL

L'embedding est enregistré dans `Cloud SQL products` (via pgvector) pour la recherche de similarité.

Workers d'Ingestion & d'Enrichissement

Worker Ingestion : De la source au Silver

- Téléchargement du dataset HuggingFace `openfoodfacts/open-prices`.
- Filtrage des données pour la France et les DOM/TOM (environ 40% conservées).
- Validation rigoureuse des lignes (prix, devise, pays, date, type de preuve).
- Normalisation des enseignes (30+ patterns regex), villes et semaines ISO.
- **Quality gates** : taux d'acceptation $\geq 60\%$, couverture enseigne $\geq 70\%$.

Sortie : BigQuery `openpricesclean`.

Worker OFF : Enrichissement via Open Food Facts

- Requête SQL pour les EANs présents dans les prix mais absents du catalogue.
- Appel à l'API `api.openfoodfacts.org/api/v2/product/{ean}`.
- Extraction d'informations clés : nom, marque, catégories (L1/L2/L3), NutriScore, NOVA, EcoScore, image.
- Téléchargement par lots dans la table BigQuery `catalogueproduits`.

Traitement des Tickets de Caisse par OCR

Notre objectif est de transformer les photos de tickets de caisse soumises par les utilisateurs en données structurées et exploitables, alimentant directement notre couche **Bronze** sans nécessiter de saisie manuelle.



Capture d'Image

Réception de la photo du ticket soumise par l'utilisateur.



Prétraitement

Normalisation et amélioration de l'image pour optimiser la reconnaissance.



Reconnaissance OCR

Application de l'OCR pour la lecture du texte sur le ticket.



Extraction Structurée

Identification et extraction des produits, prix, dates et informations du magasin.



Enjeux Techniques Majeurs

La fiabilité de ce pipeline dépend de la **qualité des images**, de la **variabilité des formats de reçus** (taille, mise en page) et de la **gestion multilingue**.

Pour adresser ces défis, nous envisageons d'utiliser des solutions robustes telles que **Tesseract** ou **Google Vision API**, potentiellement complétées par un **modèle maison** pour l'extraction spécifique.

OCR - parcours d'implémentation en quelque ligne

- **1 - Conception du package** → Architecture modulaire, développement test-based, backends OCR interchangeables
- **2 - Premier moteur : PaddleOCR** → Fonctionnel mais lourd sur CPU ; nécessite optimisations (résolution, threads, cache modèle)
 - Parser heuristique → Extraction structurée des produits, prix, dates depuis du texte brut OCR imparfait
 - Optimisations trop spécifiques → **besoin d'un modèle réellement adaptable aux multiples formats de tickets** → VLM
- **3 - Exploration VLM local** → Moondream 0.5B testé mais insuffisant sur photos de téléphone réelles
 - On veut local pour limiter l'impact infra et couts de la solution finale
- **4 - Prototype cloud opérationnel** → validation architecturale et pipeline GCP en utilisant temporairement API Groq pour OCR VLM
- **5 - Développement VLM** → Concentration sur projecteur et LoRA de l'architecture → meme entrainer seulement l'encoder du VLM est dehors des possibilités de réelle implémentation à notre niveau

OCR - notre experience en quelque mot

- On s'attendait une bonne lecture des caractères dès le départs
 - → absolument pas, souvent des petits erreurs à nettoyer avec des heuristiques de nettoyage
- On s'attendait à une structure plutot standard des tickets
 - → absolument pas, presque chaque chaine de supermarché a des format differents
 - → un erreur de formattage d'un ticket de caisse d'une chaine devient le bon formattage pour une autre
- On s'attendait une bonne performance des VLM
 - → le petit 0.5B a beacoup de difficulté quand meme sans fine tuning
 - → les grands avec API ont tendance à halluciner,
- Approche hybride API + controles rigides sur la structure attendé avant (prompt) et après (validation du json avec retry si erreurs) permettent le prototypage
- Version finale en utilisant le VLM que nous avons développé et API comme fallback
 - malheureusement encore impossible de déployer le VLM sur le mobile → meme un VLM d'environ 500M de paramètre est toujours trop gros → à évaluer aussi une étape de quantization pour limiter encore le volume et les calcules

Enrichissement Catalogue Produit

Notre catalogue produit est le cœur de l'information sur chaque référence (EAN), intégrant une multitude de données pour une analyse précise.



Validation EAN

Vérification des codes EAN-13 et EAN-8 via l'algorithme de Luhn pour assurer l'intégrité des données.



Scores Santé & Environnement

Intégration de 5 scores santé/environnement et 3 niveaux de catégories provenant d'Open Food Facts (OFF).



Image Produit

Chaque EAN est lié à une image produit, facilitant l'identification visuelle et l'expérience utilisateur.



Jointure avec les Prix

Permet des requêtes croisées entre les fiches produits et les données de prix historiques pour des analyses approfondies.



Données Visibles

Affichage transparent des informations clés : NutriScore (A-E), NOVA (1-4), et EcoScore (A-E) pour les utilisateurs.

Conclusion

Le projet a atteint son objectif →

Démontrer qu'une approche hybride en terme de sources des données et modèles utilisés peut bien fonctionner pour tracer les tendances d'inflation sur le panier spécifique utilisateur

Merci pour votre attention !

